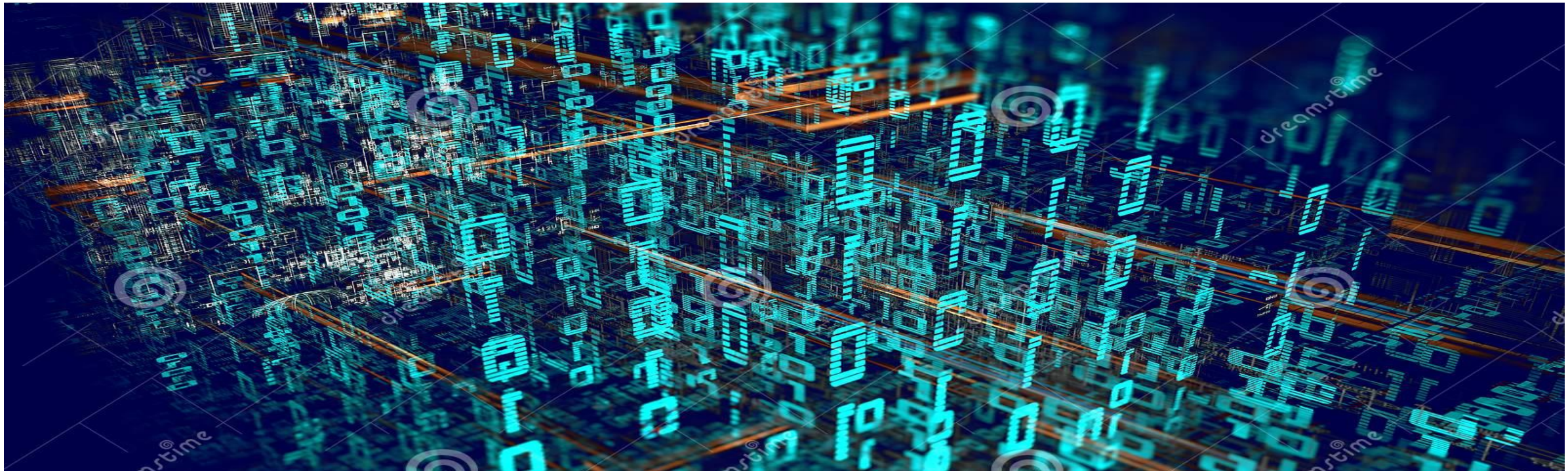
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



MODULE 4

INTERMEDIATE CODE GENERATION

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Procedures

- The procedure is such an important and frequently used programming construct that it is imperative for a compiler to generate good code for procedure calls and returns.
- In three-address code, a function call is unraveled into the evaluation of parameters in preparation for a call, followed by the call itself.

Suppose that a is an array of integers, and that f is a function from integers to integers. Then, the assignment

```
n = f ( a [ i ] ) ;
```

f is a **function**
(**a [i]**) is the **parameter** of the function

Three-address code:

```
1)  t1 = i * 4
2)  t2 = a [ t1 ]
3)  param t2
4)  t3 = call f, 1
5)  n = t3
```

- The **first two lines** compute the value of the expression **a [i]** into temporary t_2
- **Line 3** makes t_2 an **actual parameter** for the call on **line 4** of **f** with one parameter.
- **Line 5** assigns the value returned by the function call to t_3

Adding functions to the source language

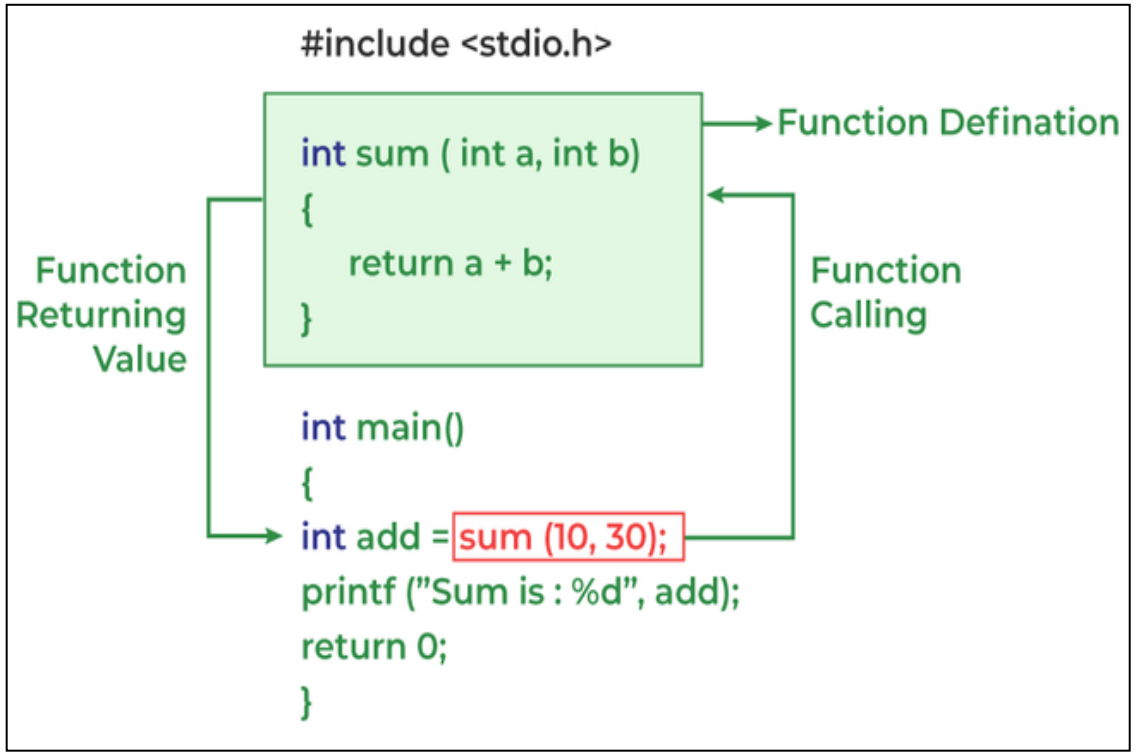
- The productions shows the function definitions and function calls

```
D → define T id ( F ) { S }
F → ϵ | T id, F
S → return E;
E → id ( A )
A → ϵ | E, A
```

- Nonterminals **D** generate ***declaration***
- Nonterminals **T** generate ***types***
- Nonterminals **S** generate ***statements***
- Nonterminals ***E*** generate ***expressions***

- A function definition generated by ***D*** consists of **keyword** ***define***, a return type **(T)**, the function name **(id)**, formal parameters (**F**) in parentheses and a function body consisting of a statement (**S**).
- Nonterminal **F** generates ***zero or more formal parameters***, where a formal parameter consists of a type (**T**) followed by an identifier (**id**).
- The production for **S** adds a statement that returns the value of an expression.
- The production for **E** adds function calls, with actual parameters generated by ***A***.
- An actual parameter is an expression.

```
D → define T id ( F ) { S }
F → ϵ | T id, F
S → return E;
E → id ( A )
A → ϵ | E, A
```



float add( ) // **F→ε**

{ return add () }

float add (int a) //***F→T id*** (id is variable name)

{ return add(a) }

float add(int a, float b) // ***F → ε| T id , F***

D → define T id (F) { S }

float // T (datatype)

add ( )//id (function name)

F → ε| T id , F

int / float //T

a // id is variable

S → return E;

E →id(A) ;

A→ ε | E, A

id represent function name

A represent actual parameter

Assignment Statements

- In the **Syntax Directed Translation**, assignment statement is mainly deals with ***expressions***.
- The expression can be of type ***real***, ***integer***, ***array*** and ***records***.
- Suppose the context in which an assignment appears is given by the grammar

  ➢ Structure of a **procedural programming language (Pascal,C or Ada)**

P → MD
M → ε
D → D; D| id: T | proc id; N D;S
N → ε

D → D; D | id: T | proc id; N D; S

- This is the rule for declarations (D), and it allows for:
  - ***Sequences of declarations*** (**D; D**), allowing multiple declarations separated by semicolons.
  - ***Variable declaration*** (**id: T**), where id is an identifier and T is a type.
  - ***Procedure declaration*** (**proc id; N D; S**), where
    - **id** is a procedure name,
    - **N** is an optional part (which is empty in this case)
    - **D** is a block of declarations
    - **S** represents the statements within the procedure.

Example

```
proc calculate;
    num: int;
    result: int;
    begin
        result := num * 10;
    end;
```

$P \rightarrow MD$
$M \rightarrow \varepsilon$
$D \rightarrow D; D \mid id: T \mid proc\ id; N\ D; S$
$N \rightarrow \varepsilon$

D → proc id; N D; S:

proc calculate; id = calculate
N is epsilon (empty), proceed to D

D → id: T (Declarations inside the procedure)

num: int;
result: int;

S (Statements inside the procedure)

```
begin
    result := num * 10;
end;
```

- Consider the grammar

$$
\begin{aligned}
S &\rightarrow id := E \\
E &\rightarrow E1 + E2 \\
E &\rightarrow E1 * E2 \\
E &\rightarrow (E1) \\
E &\rightarrow id
\end{aligned}
$$

Function	Description
lookup()	Searches particular variable or identifier in symbol table
emit()	Generates 3 address code
newtemp()	Used to generate new temporary variables (t1,t2,..)
E.place	holds the value of E (tells about the name that will hold the value of the expression). place is an attribute of E.

Production	Semantic actions
S → id :=E	{ p = **lookup**(id.name); If p ≠ nil then **emit** (p = E.place) else Error; }
E →E1 + E2	{ E.place = newtemp(); emit (E.place = E1.place '+' E2.place) }
E → E1 * E2	{ E.place = newtemp(); emit (E.place = E1.place '*' E2.place) }
E → -E1	{ E.place : = newtemp; emit (E.place ‘: =’ ‘uminus’ E1.place) }
E → (E1)	{ E.place = E1.place }
E → id	{ p = **look_up**(id.name); If p ≠ nil then **emit** (E.place=p) else Error; }

- ***lookup*** check whether identifier (id.name) is present in symbol table. If present then that value will be assigned to p.
- If p is not equal to null, then emit, which will create a three address code and assign it to p.

E.place = t1
t1=E1+E2

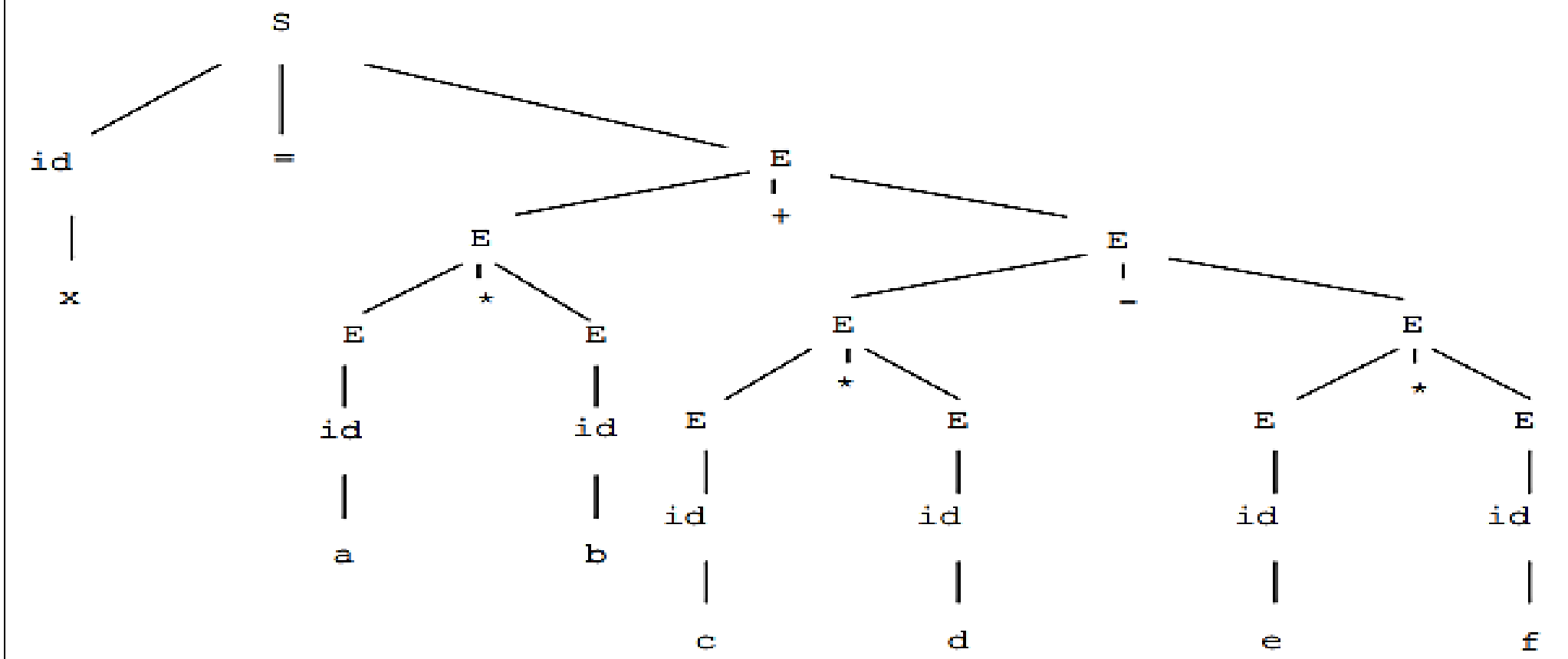
E.place = t2
t2=E1*E2

E.place = t3
t3=-E1

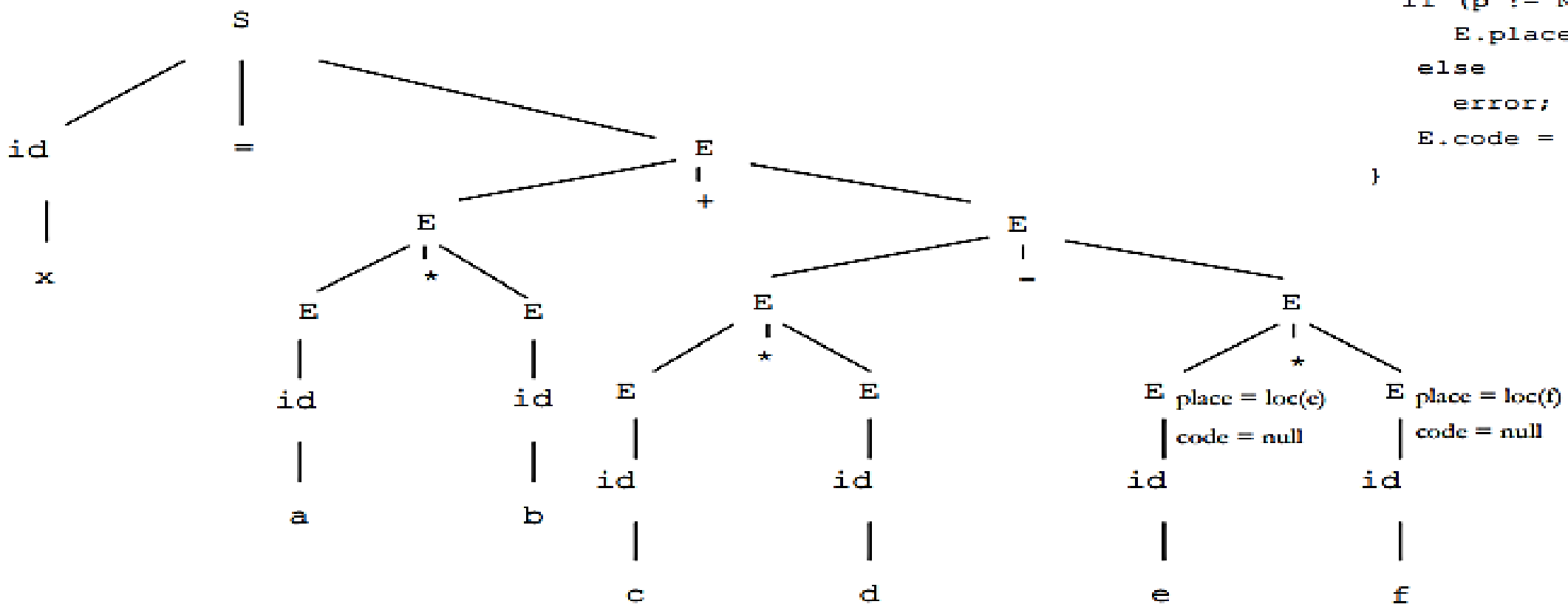
- Whenever id is present, check for symbol table.
- If p is null then return the error value

Assignment: Example

```
x = a * b + c * d - e * f;
```



```
x = a * b + c * d - e * f;
```



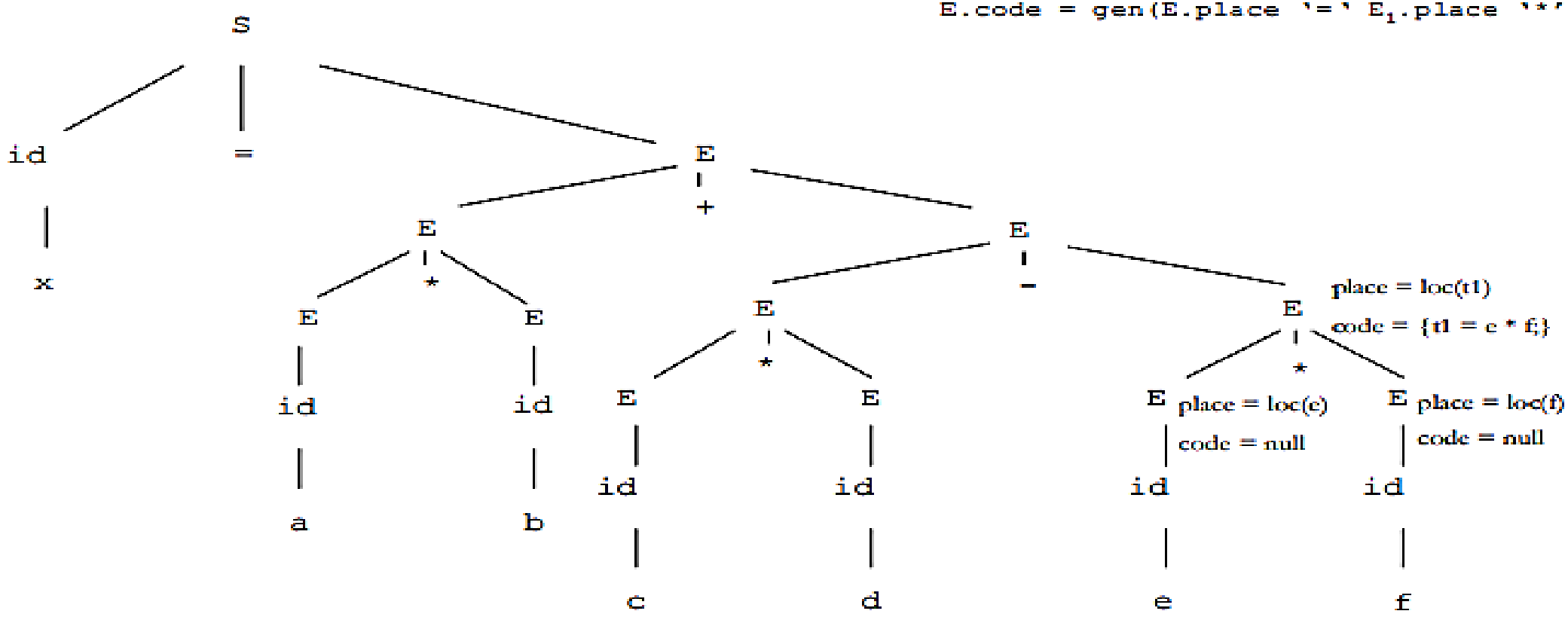
Production:

```
E → id        { p = lookup(id.name);
                if (p != NULL)
                   E.place = p;
                else
                   error;
                E.code = null list;
              }
```

```
x = a * b + c * d - e * f;
```

Production:

$E \rightarrow E_1 * E_2$ {E.place = newtemp();
E.code = gen(E.place '=' E_1.place '*' E_2.place);}

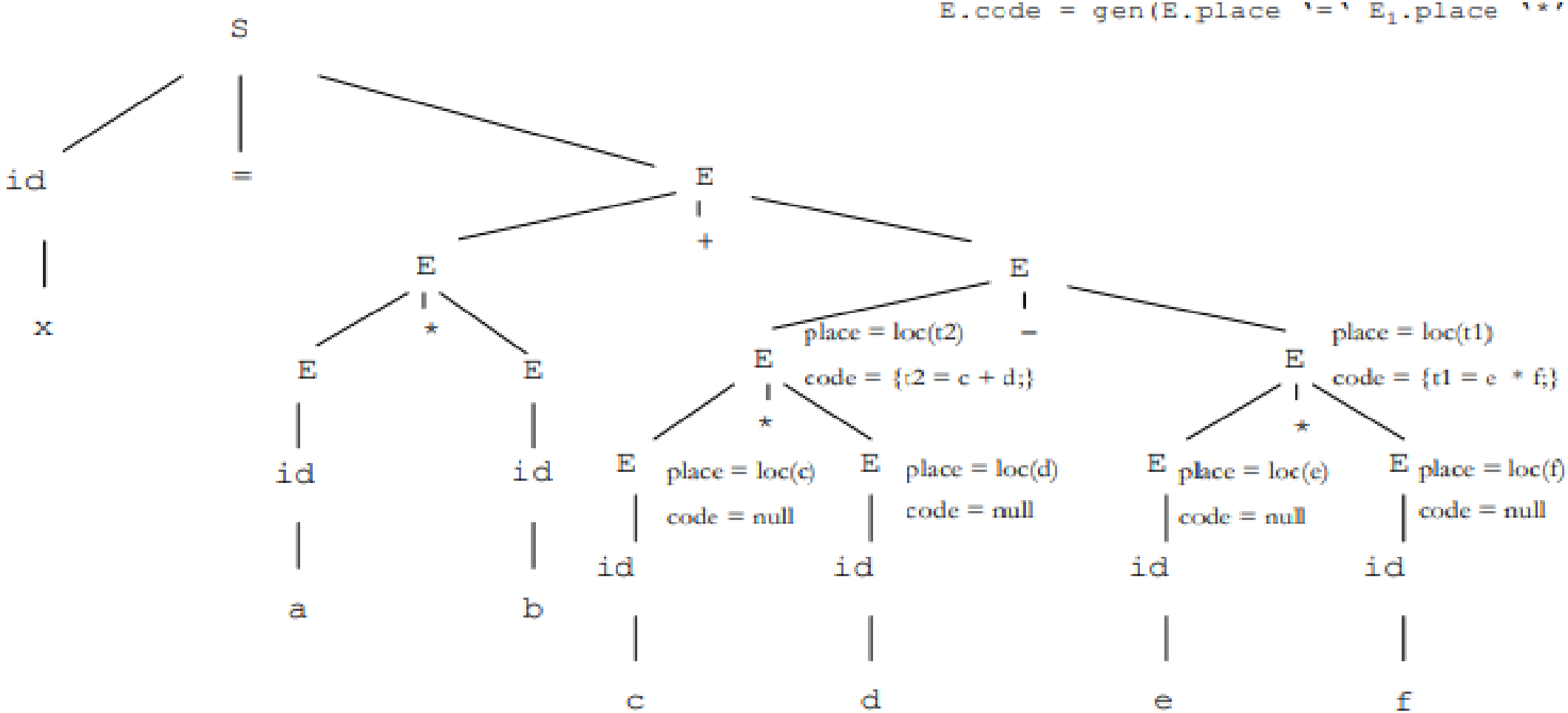


x = a * b + c * d - e * f;

Production:

E → E_1 * E_2 {E.place = newtemp();

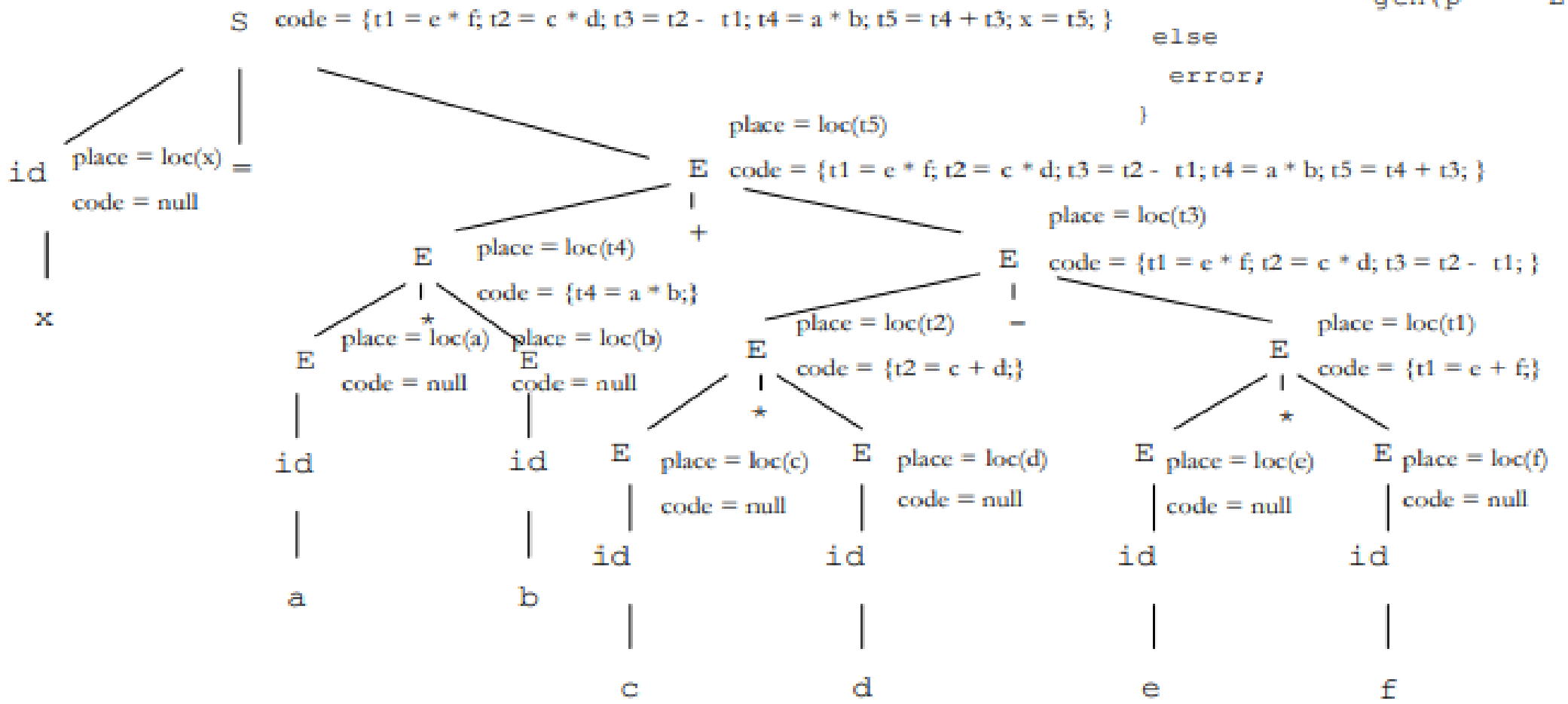
E.code = gen(E.place '=' E_1.place '*' E_2.place);}



Production:

```
S → id = E    { p = lookup(id.name);
                if (p != NULL)
                  E.code = append(E.code,
                                  gen(p '=' E.place));
                else
                  error;
              }
```

```
x = a * b + c * d - e * f;
```



Translation of Expressions

1. Operations Within Expressions

- The syntax-directed definition builds up the ***three-address code for an assignment statement*** **S**
- Attribute **code** for assignment statement **S**
- Attributes **addr** and **code** for an expression **E**
- Attributes **S.code** and **E.code** denote the ***three-address code*** for **S** and **E**.
- Attribute **E.addr** denotes the address that will hold the value of **E**
  - Can be a name, a constant, or a temporary
- **top** denote the ***current symbol table***.
- **top.get()** retrieves the entry.
- **newtemp()** allocates and returns a new temporary symbol
- **gen()** – Helper function to create a 3AC instruction

```
If id1.lexeme = x, id2.lexeme =y and id3.lexeme = z:
gen (id1.lexeme, ':=', id2.lexeme, '+', id3.lexeme)
```

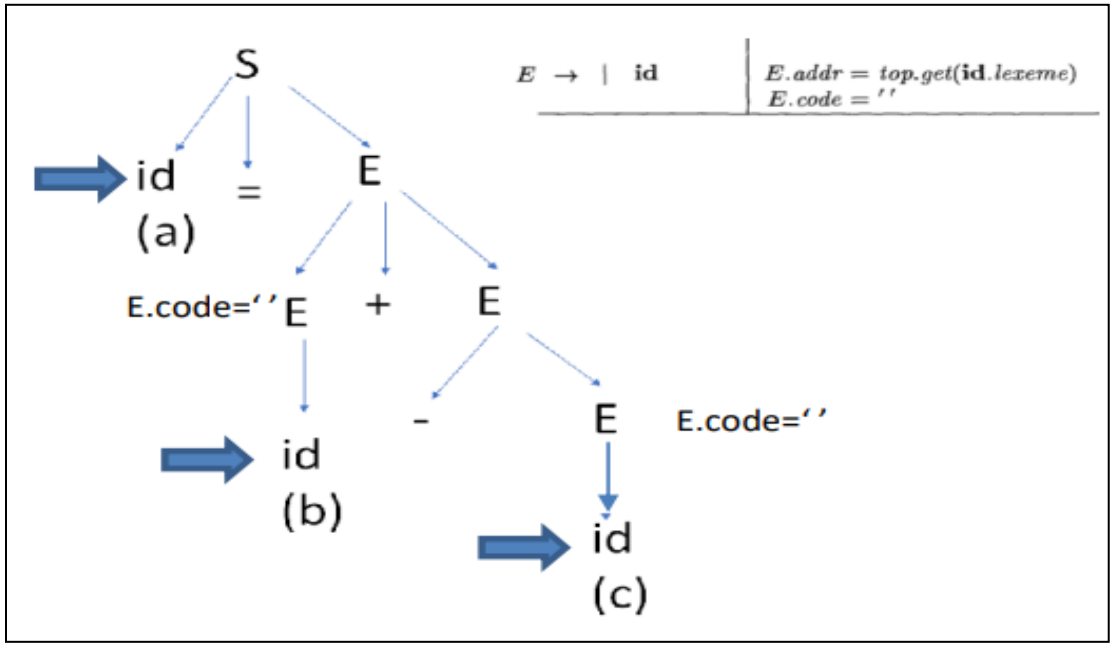
It will produce three address code **x:=y+z**

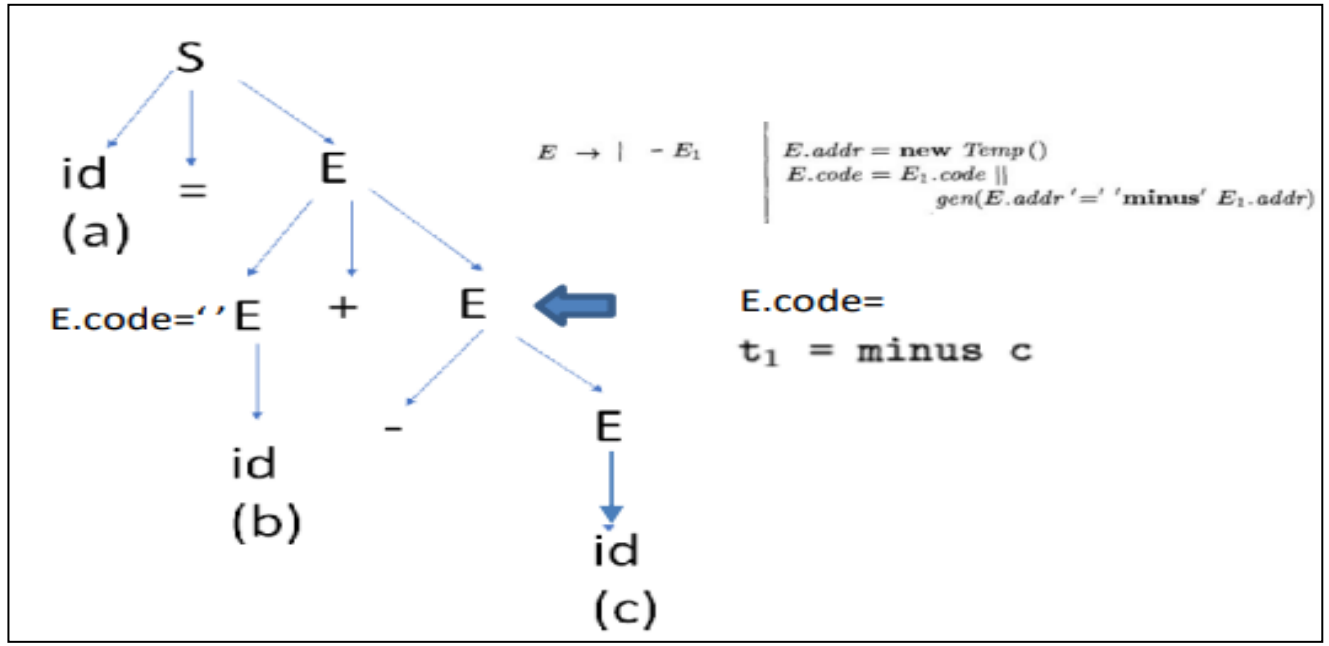
Three-address code for expressions

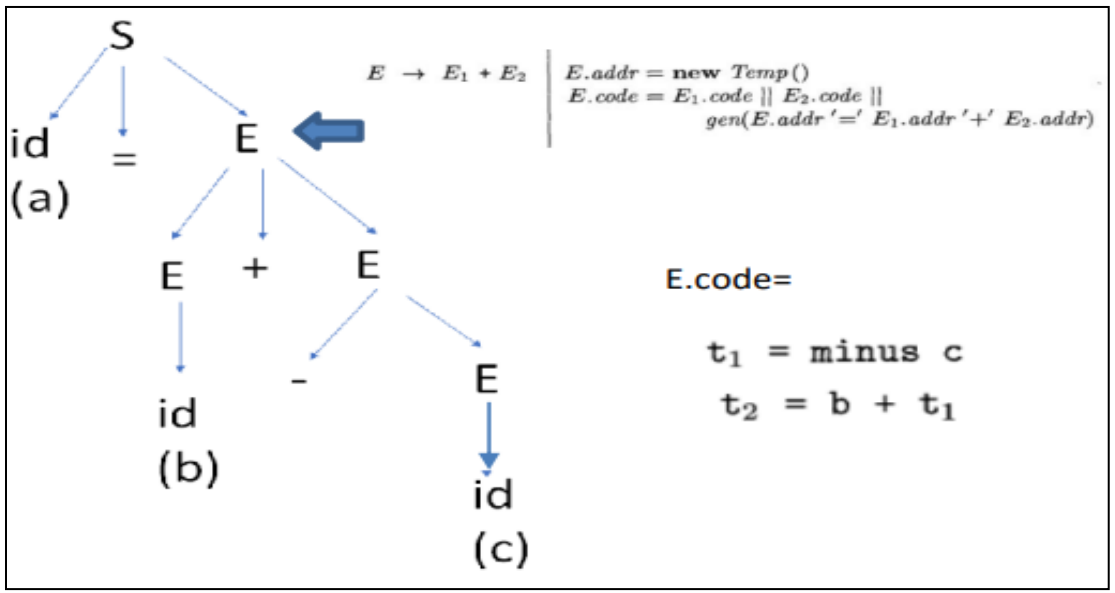
Production	Semantic Rules
$S \rightarrow$ id = E	$S.code = E.code$ || $gen(top.get(\text{id}.lexeme)$ "=" $E.addr)$
$E \rightarrow E1 + E2$	$E.addr =$ new $Temp()$ $E.code = E1.code \,||\, E2.code$ || $gen(E.addr$ "=" $E1.addr$ " + " $E2.addr)$
$E \rightarrow E1 * E2$	$E.addr =$ new $Temp()$ $E.code = E1.code \,||\, E2.code$ || $gen(E.addr$ "=" $E1.addr$ " * " $E2.addr)$
$E \rightarrow -E1$	$E.addr =$ new $Temp()$ $E.code = E1.code$ || $gen(E.addr$ "=" " – "$E1.addr)$
$E \rightarrow (E1)$	$E.addr = E1.addr$ $E.code = E1.code$
$E \rightarrow$ id	$E.addr = top.get(\text{id}.lexeme)$ $E.code =$ ""

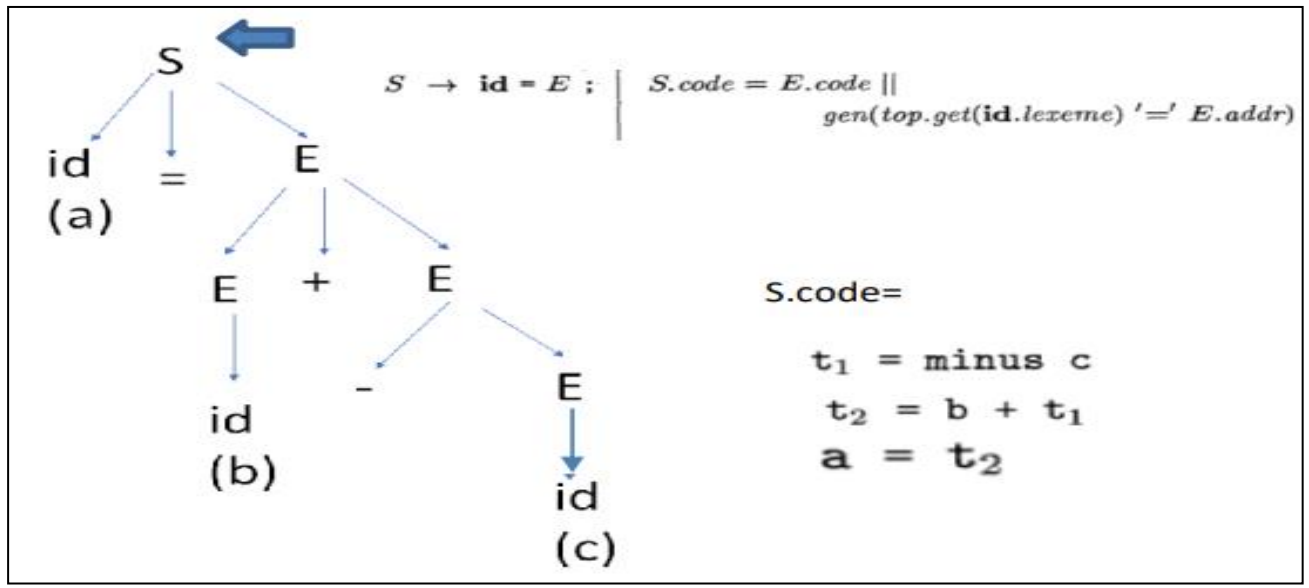
- When we translate **E -> E1+E2**, the semantic rules combine E1.code, E2.code, and an instruction that adds the values of E1 and E2 to create E.code.
  - The instruction saves the result of the addition as E.addr, a new temporary name for E.
- E → - E1 has a similar translation.
  - The rules provide instruction to do the unary minus operation and build a new temporary for E.
- S→ id = E; creates instructions that assign the expression E's value to the identifier id.
  - The semantic rule for this production uses function *top.get* to determine the address of the identifier represented by **id,** as in the rules for *E* —*v* **id.**
  - *S.code* consists of the instructions to compute the value of *E* into an address given by *E.addr*, followed by an assignment to the address *top.get(id.lexeme)* for this instance of **id.**

- **Example**: Translates the assignment statement **a = b + - c** into the three-address code sequence based on syntax-directed definition









2. Incremental Translation

- Code attributes can be long strings, so they are usually generated incrementally.
- In incremental translation, generate only the new three address instructions.
- The syntax-directed specification generates the same code as the translation scheme for assignment statement .
- **E.code** attribute is **not used**, since there is a single sequence of instructions that is created by successive calls to **gen()**.

- **gen()** not only constructs a three-address instruction, it appends the instruction to the sequence of instructions generated so far.

Production	Semantic Rules
$S \rightarrow$ id = E	$gen(top.get(\text{id}.lexeme)$ "=" $E.addr)$
$E \rightarrow E1 + E2$	$E.addr$ = new $Temp()$ $gen(E.addr$ "=" $E1.addr$ " + " $E2.addr)$
$E \rightarrow E1 * E2$	$E.addr$ = new $Temp()$ $gen(E.addr$ "=" $E1.addr$ " * " $E2.addr)$
$E \rightarrow -E1$	$E.addr$ = new $Temp()$ $gen(E.addr$ "=" " – "$E1.addr)$
$E \rightarrow (E1)$	$E.addr = E1.addr$
$E \rightarrow$ id	$E.addr = top.get(\text{id}.lexeme)$

Control Flow

- Control flow can be defined as the ***order in which statements are executed or evaluated.***
- Control flow statements include ***branching statements*** and ***looping statements***.
- This control flow can be decided with the help of a ***boolean expression***(an expression that evaluates to either true or false).
- The **translation of boolean expressions** is connected to the **translation of statements** like
  - ***if-else statements***
  - **while statements**
- **Boolean expressions** are frequently used in programming languages to

(i) Alter the flow of control

- Boolean expressions are used as conditional expressions in statements that alter the flow of control.
- Example :

if(E) S **E** is a ***boolean expression,*** **S** is the ***statement***

Control will reach S only if the E evaluates to true.

(ii) Compute the logical values

- A Boolean expression can represent ***true* or *false*** as values.
- These Boolean phrases can be evaluated using ***three-address instructions*** and **logical operators (arithmetic expression)**

1. Boolean Expressions

- Boolean expressions are made of **Boolean operators** (&&(AND), ||(OR), !(NOT)) applied to the elements that are ***Boolean variables*** or ***relational expressions.***
- Relational expressions are of the form

E1 rel E2

E1 and **E1** are **arithmetic expressions**

- Consider the boolean expressions generated by the following grammar:

B → B || B | B && B | !B | (B) | E rel E | true | false

- The **attribute** ***rel.op*** is used to indicate the operator represented by ***rel*** among the *6* ***comparison operators***

<, <=, =, !=, > *or* >=

- Assume **||** and **&&** are **left-associative** and **||** has the **lowest precedence**, then **&&** and finally **!**
- Given an expression

B1 || B2

- If we determine that B1 is true then we can conclude the entire expression if true without evaluating B2.

B1 && B2

- If B1 is false, the entire expression is false.
- If B1 is true then only check for B2, if B2 is also true, only then the expression is true.

2. Short-Circuit Code

- In **short-circuit (or jumping) code**, the Boolean operators **&&**, **II**, and **!** ***translate into jumps***.
- The operators themselves do not appear in the code.
- The **value of a Boolean expression** is represented by a ***position in the code sequence.***

Example:

- Consider the statement

```
If (x < 100 || x > 200 && x != y)
x = 0;
```

- The statement is translated into the following code segment

```
    if x < 100 goto L2
    if False x > 200 goto L1
    if False x != y goto L1
L2: x = 0
L1:
```

- First the expression **x < 100** is checked if its **true** control will directly jump to label L2 and expression x=0 will be executed.
- If **x < 100** comes to **false**, then the other expression will be checked part-wise.

- If **x > 200** is true, then the remaining part of the expression is checked (x!=y), if it is false the whole expression again will become false and control will jump to label L1.
- Else if both expressions(x> 200 && x!=y) are true, then the control will jump to label L2.

3. Flow-of-Control Statements

- Consider the translation of Boolean expressions into three-address code in the context of statements generated by the following grammar

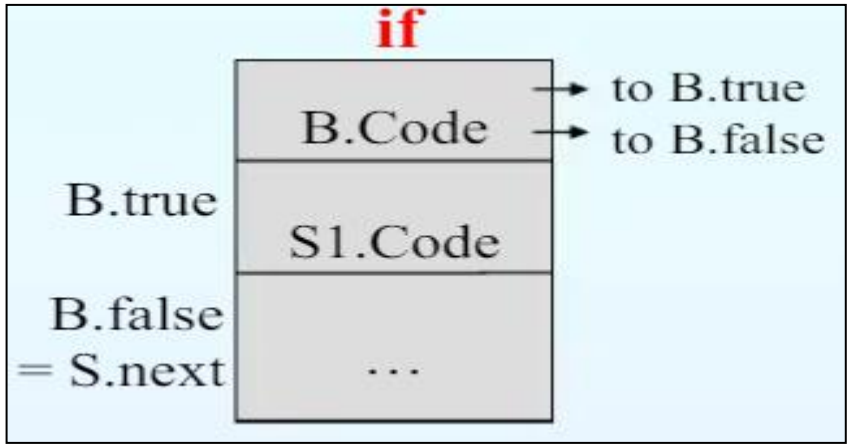
S → if (B) S1
S → if (B) S1 else S2
S → while (B) S1

- Nonterminal **B** represents a ***Boolean expression***
- Non-terminal **S** represents a ***statement***.

- **B** and **S** have a ***synthesized attribute code***, which gives the **translation into three-address instructions.**
- Using **syntax-directed definitions** we build up the translations ***B.code*** and ***S.code*** as strings.
- **Inherited attributes (*B.true, B.false, S.next*)** are used to manage the labels for the jumps in ***B.code*** and ***S.code***
- With a boolean expression *B*, we associate two labels:
  - **B.true**, the label to which control flows if B is true
  - **B.false**, the label to which control flows if B is false.
- ***S.next*** that denotes a label for the instruction immediately following the code for *S*

- **The translation of** **if (B) S1** consists of B.code followed by S1.code

Code for control flow statements



- B.code is the synthesized attribute for B
- B.code are jumps based on the value of B
- If B is true, control flows to the first instruction of S1.code.
- If B is false, control flows to the instruction immediately following S1 . code.

SDD – Semantic Rule

```
{
B.true = newlabel();
B.false = S.next;
S1.next = S.next;
S.code = B.code
          || label(B.true ) ||  S1.code;
}
```

- If expression B is true, create a newlabel for it.*(used to jump to the correct block of code if B evaluates to true)*
- If expression B is false , then come out of the statement (S.next)
- End of the statement of S.code will be S.next.
- S1.next ensures that after executing S1, control goes to the next part of the program

newlabel - returns a new symbolic label each time it is called

Example

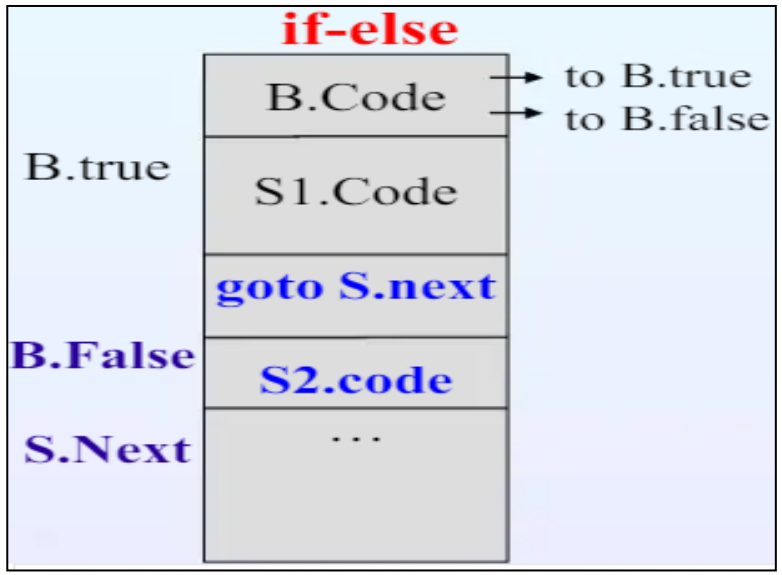
1. If (a < b) S1

- Assume that the attributes true and false exist for the entire expression as labels Ltrue and Lfalse respectively.
- Then the translation is

```
        if a < b goto Ltrue
        goto Lfalse
Ltrue:  S1.code
Lfalse:
```

- **The translation of** **S → if (B) S1 else S2**

Code for control flow statements



- If B.code is true then a new label B.true is created and statement S1.code is executed
- If B.code is false then a new label B.false is created and statement S2.code is executed.
- Then the remaining code S.next will be executed.

- After S1.code, S.next will be executed as well as after S2.code, S.next will be executed

SDD – Semantic Rule

```
B.true = newlabel();
B.false = newlabel();
S1.next = S.next
S2.next = S.next;
S.code = B.code
            || label(B.true) ||S1.code
            || gen('goto' S.next)
            || label(B.false) || S2.code;
```

gen ( ) – generates the code (string) passed as a parameter to it

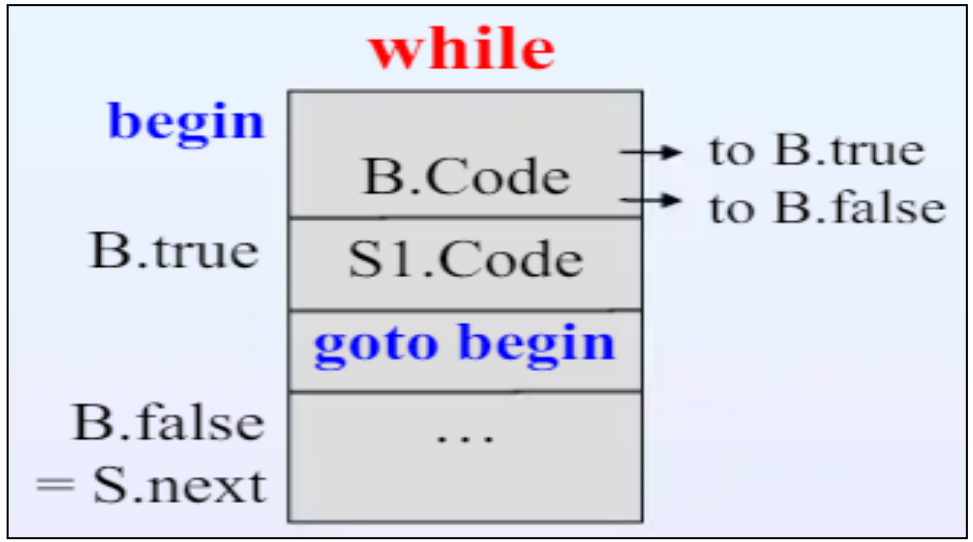
Example

2. If (a < b or c < d) S1 else S2

```
        if a < b goto Ltrue
        goto L1
   L1:  if c < d goto Ltrue
        goto Lfalse
Ltrue:  S1.code
         goto Snext
Lfalse: S2.code
Snext:
```

- **The translation of** **S → while (B) S1**

Code for control flow statements



- If B.code is true then S1.code will be executed and control will again jump to label begin and the condition for B.code will be checked.
- It is done until B.code becomes false and then the remaining code will be executed (control will jump to label B.false).

SDD – Semantic Rule

begin = newlabel()
B.true = newlabel()
B.false = S.next
S1.next = S.begin
S.code = label (begin) || B.code
|| label (B.true) ||S1.code
|| gen('goto' begin)

begin – contains the label / address pointing to the beginning of the code chunk for the statement generated (if any) by the non terminal

Example

```
while a < b do
    if c < d then
        x := y + z
    else
        x := y − z
```

```
L1: if a < b goto L2
    goto Lnext
L2: if c < d goto L3
    goto L4
L3: t1 := y + z
    x := t1
    goto L1
L4: t2:= y − z
    x:=t2
    gotoL1
Lnext:
```

Syntax-directed definition for flow-of-control statements.

- A program consists of a statement generated by **P -> S**. This production initialize S.next to a new label. P.code consists of S.code followed by the new label S.next.
- Token assign in the production **S —> assign** is a placeholder for assignment statements.
- Code for ***S →S1 S2*** consists of code for *S1* that is followed by code for *S2*.
- The first instruction after S1 is the beginning of the code for S2, and the instruction after code S2 code, is the instruction after code for *S*.

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

4. Control-Flow Translation of Boolean Expressions

- A boolean expression is translated into three address instructions that evaluate the expression using create labels only when they are needed.
- A boolean B is translated into three-address instructions that evaluate B using conditional and unconditional jumps to one of two labels, these are *B.true* and *B.false*.

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1$ **rel** E_2	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen(\texttt{'if'}\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ \texttt{'goto'}\ B.true)$ $\mid\mid gen(\texttt{'goto'}\ B.false)$
$B \rightarrow$ **true**	$B.code = gen(\texttt{'goto'}\ B.true)$
$B \rightarrow$ **false**	$B.code = gen(\texttt{'goto'}\ B.false)$

- ***B→ B1 || B2***
  - If B1 is true, then the B itself is true, so B1.true is the same as B.true.
  - If B1 is false, then B2 must be evaluated, for evaluation we have to generate three address code for that we call the function newlabel(). So we make B1.false be the label of the first instruction in the code for B2.

```
B1.true = B.true;
B1.false = newLabel();
B2.true = B.true;
B2.false = B.false;
B.code = B1.code ||
         label(B1.false) ||
         B2.code;
```

  - If B1 is false and B2 is true, then B is true (as per logical operator)
  - If B1 is false and B2 is false, then B is false

- ***B→ B1&& B2***
  - If B1 is true, then we evaluate B2 for that we have to generate three address code for that we call the function newlabel().
  - If B1 is false, then no need to evaluate B2 and B is also false.
  - If B1 is true and B2 is true then B is true
  - If B1 is false and B2 is true then B is fale

```
B1.true = newLabel();
B1.false = B.false;
B2.true = B.true;
B2.false = B.false;
B.code = B1.code ||
         label(B1.true) ||
         B2.code;
```

- ***B→ ! B1***
  - If B1 is true, then B will become false
  - If B1 is false, then B will become true

```
B1.true = B.false;
B1.false = B.true;
B.code = B1.code;
```

- ***B→ E1 rel E2***
  - Translated directly into a comparison three-address instruction with jumps to the appropriate places.
  - If we have *B* with the form of *a* < *b*. It translates to

```
B.code = E1.code || E2.code ||
         gen('if' E1.addr rel.op E2.addr
               'goto' B.true) ||
         gen('goto' B.false)
```

if a < b goto B.true
goto B.false

Self Study Topic

5. Avoiding Redundant Gotos

6. Boolean Values and Jumping Code